

SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, Song Han

ICML 2023

딥러닝 논문 읽기 모임 편디멘털
오영화, 차지윤

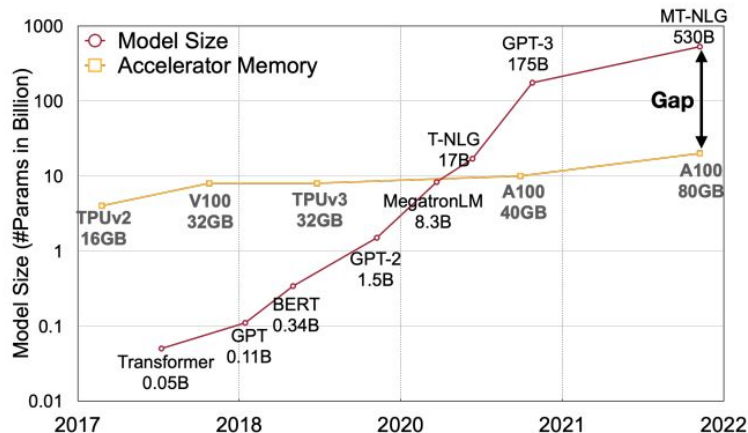
Introduction

1. Large Language Model

- GPT-3 모델은 175B 파라미터가 있고 이는 FP16에서 최소 360GB의 메모리를 차지한다.
- 8×48GB A6000 GPUs 나 5×80GB A100 GPUs 정도가 겨우 추론을 위해 사용될 수 있을 만큼, 단일 GPU나 서버로는 엄청난 크기의 weight와 activation을 처리하기 힘들다

2. Large Language Model Quantization

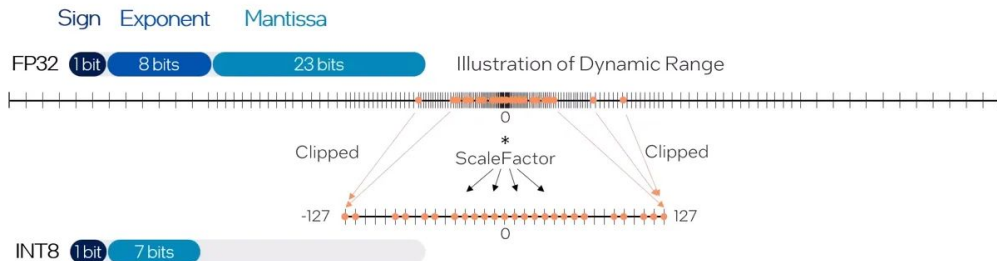
- 양자화는 가중치와 활성화 값을 낮은 정밀도의 숫자로 변환하여 메모리를 절약하고 (LLM의 사이즈를 줄여서) 연산 속도를 높이는 (빠른 추론) 기술
- **가중치 (Weight):** 딥러닝 모델의 학습 파라미터
- **활성화 (Activation):** 모델 레이어 간에 전달되는 값
- 낮은 정밀도 숫자 변환: e.g. fp16->int8



Background

3 type of model quantization

1. **훈련 후 양자화** Post-Training Quantization, PTQ
 - 사전 훈련된 모델의 가중치와 활성화 값을 낮은 정밀도로 변환
 - GPU에서 선호, CPU에선 사용되지 않음
 - 장점 : 추론 속도 상승, *단점 : 추가적인 단계 필요 및 정확도 손실 발생*
2. **훈련 중 양자화** Quantization-Aware Training, QAT
 - 모델이 훈련하는 동안 양자화를 고려하여 가중치와 활성화 값을 조정
 - 훈련된 모델이 양자화에 민감하게 반응
 - 장점 : PTQ보다는 정확도 감소 폭을 최소화 *단점 : 양자화 고려 훈련이라 구현이 복잡함*
3. **추론 중 활성화 양자화** Dynamic Range Quantization
 - 모델의 가중치 양자화이지만 활성화에 대한 계산을 진행하기 전까지는 양자화 진행하지 않음
 - 장점 : 추론 중에 메모리 사용량 줄이고 추론 속도 상승 *단점 : 특정 모델에 따른 성능차이 발생*



Background

What is per-tensor/token/channel quantization?

- Traditional Quantization, Min/Max Quantization

$$S = \frac{X_{\max} - X_{\min}}{2^k} \quad Z = -\text{round} \left(\frac{X_{\max} + X_{\min}}{2S} \right)$$

- Per-tensor Quantization**

: 전체 행렬 (모든 가중치와 활성화) 에 대해 단일 스텝 크기
: 구현이 간단함 (제일 정확도가 떨어짐)

- Per-Token Quantization**

: 토큰 (특정 위치의 활성화 값) 에 대해 다른 스텝 크기
: NLP에서 각 단어의 임베딩 값을 다른 양자화 크기 변환

- Per-channel Quantization**

: 가중치 출력 채널에 대해 다른 양자화 스텝 크기
: CNN에서 각 필터의 가중치를 다른 양자화 크기 변환

=> Activation의 경우 **high-throughput 연산에 부적절**

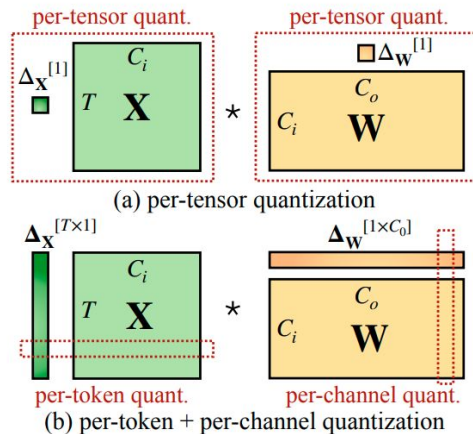
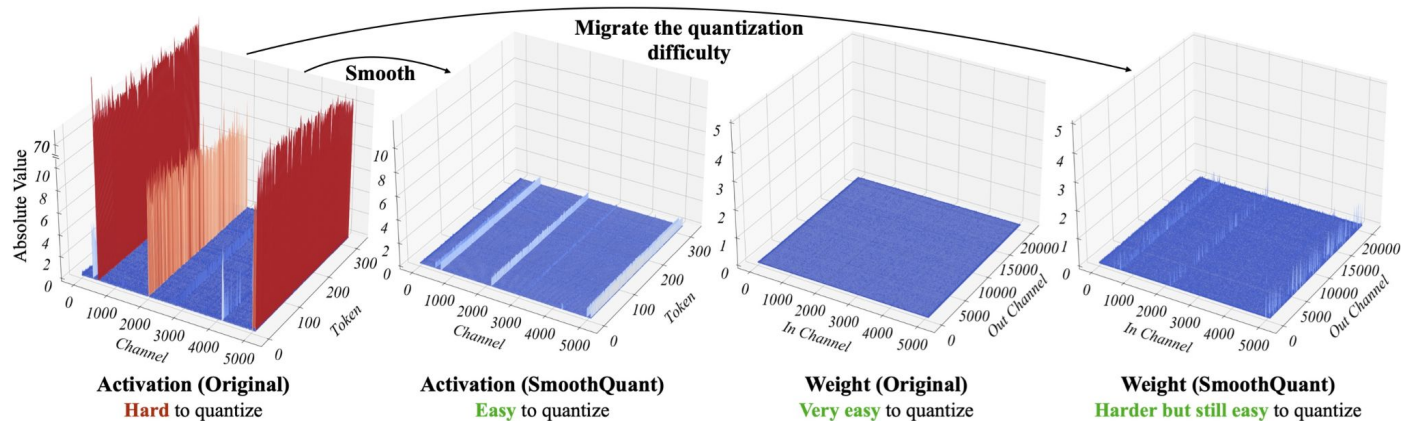


Table 1: Among different activation quantization schemes, only per-channel quantization (Bondarenko et al., 2021) preserves the accuracy, but it is *not* compatible (marked in gray) with INT8 GEMM kernels. We report the average accuracy on WinoGrande, HellaSwag, PIQA, and LAMBADA.

Model size (OPT-)	6.7B	13B	30B	66B	175B
FP16	64.9%	65.6%	67.9%	69.5%	71.6%
INT8 per-tensor	39.9%	33.0%	32.8%	33.1%	32.3%
INT8 per-token	42.5%	33.0%	33.1%	32.9%	31.7%
INT8 per-channel	64.8%	65.6%	68.0%	69.4%	71.4%

Review of Quantization Difficulty

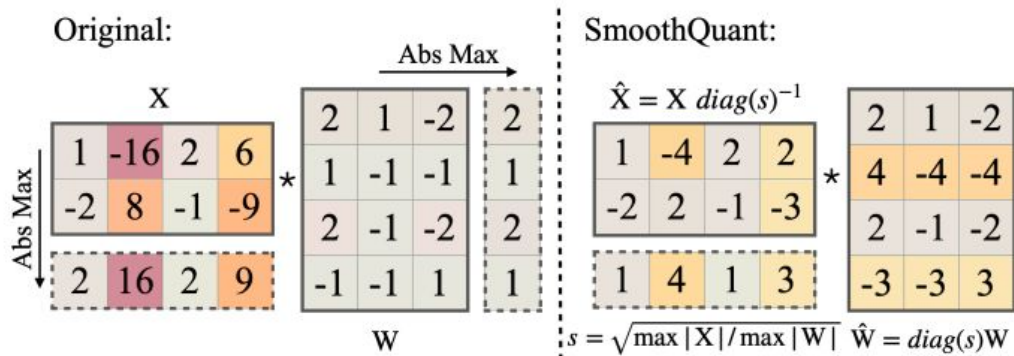
Why quantization is difficult?



- **Activations are harder to quantize than weights**
: Weight의 분포가 더 uniform, flat하다
- **Outliers make activation quantization difficult (in per-token Quantization)**
: 대부분의 activation 값보다 outlier의 값이 100배 이상 크다.
- **Outliers persist in fixed channels**
=> **activation**에서 아웃라이어가 같은 채널에서는 변동이 작으니,
token, tensor 별 양자화보다 **error**가 적을 것이다!

SmoothQuant

“smooth” the input activation by dividing it by a per-channel smoothing factor $s \in \mathbb{R}^{C_i}$



- input X : 행은 토큰, 열은 채널 방향 (linear norm)
- SmoothQuant는 X 의 outlier 값을 줄여서 swing을 줄여 평탄화 W 의 특성을 살림

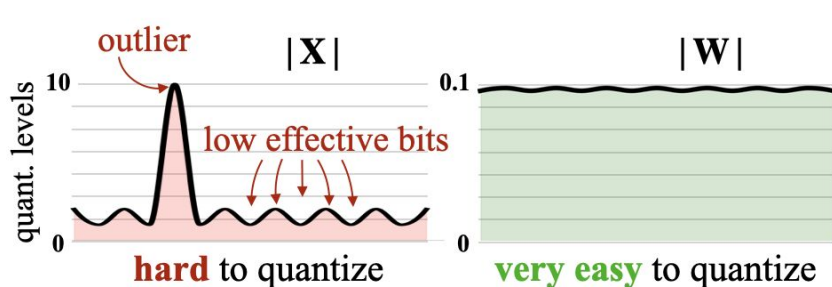
$$Y = \text{diag}(\Delta_X^{\text{FP16}}) \cdot (\bar{X}^{\text{INT8}} \cdot \bar{W}^{\text{INT8}}) \cdot \text{diag}(\Delta_W^{\text{FP16}})$$

SmoothQuant

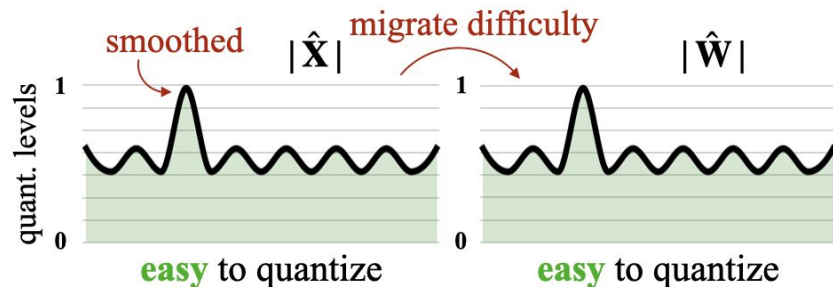
Migrate the quantization difficulty from activations to weights.

$$s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}$$

- X 의 max값이 클 수록 scale을 많이 해주고 가중치의 max가 작을수록 scale이 크다
- **migration strength α** : 얼마나 activation을 weight로 옮길지 결정하는 하이퍼파라미터 (논문에서는 0.5)



(a) Original

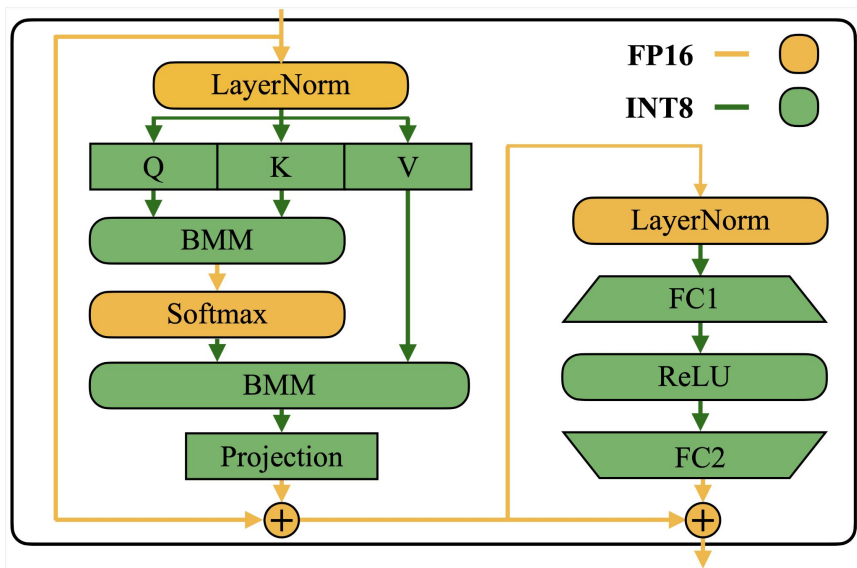


(b) SmoothQuant (ours)

SmoothQuant

Applying SmoothQuant to Transformer blocks.

- Layer 정규화, Relu, Softmax와 덧셈같이 가벼운 계산은 **FP16**
- Attention의 Query, Keyword, Value, Batched Matrix Multiplication과 Projection, Fully Connected Layer같은 computing 자원이 필요한 계산은 **Int8**

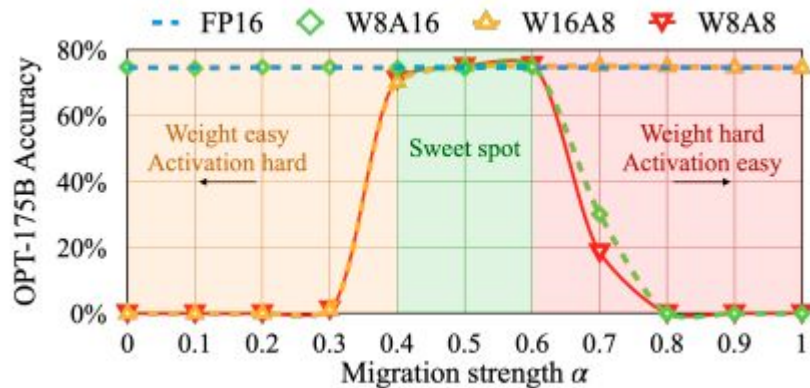


Experiment

Experiment Setting

Table 2: Quantization setting of the baselines and SmoothQuant. All weight and activations use INT8 representations unless specified. For SmoothQuant, the efficiency **improves** from O1 to O3 (i.e., lower latency).

Method	Weight	Activation
W8A8	per-tensor	per-tensor dynamic
ZeroQuant	group-wise	per-token dynamic
LLM.int8()	per-channel	per-token dynamic+FP16
Outlier Suppression	per-tensor	per-tensor static
SmoothQuant-O1	per-tensor	per-token dynamic
SmoothQuant-O2	per-tensor	per-tensor dynamic
SmoothQuant-O3	per-tensor	per-tensor static



- W8A8 : Weight quantized 8 bit, Activation quantized 8 bit
- group-wise Quantization : 각 채널 그룹에 대해 다른 양자화 단계 사용
- O1: per-token dynamic activation quantization
- O2: per-tensor dynamic activation quantization
- O3: per-channel static activation quantization

Experiment

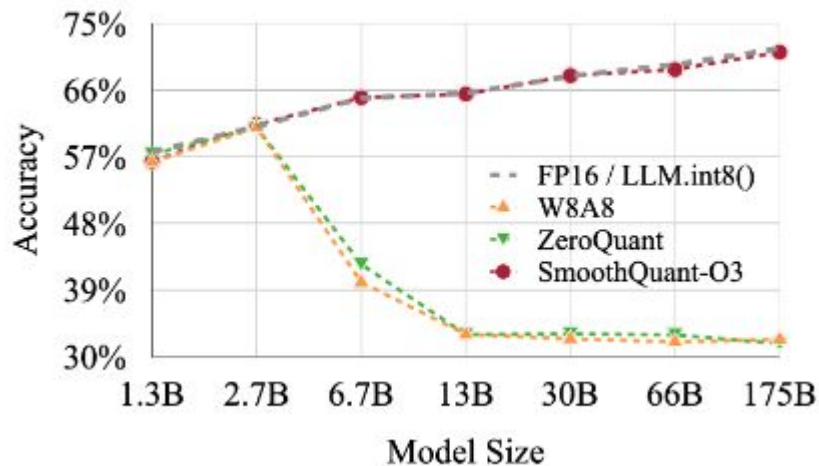
1. Accuracy of zero-shot tasks with OPT models

<i>OPT-175B</i>	LAMBADA	HellaSwag	PIQA	WinoGrande	OpenBookQA	RTE	COPA	Average↑	WikiText↓
FP16	74.7%	59.3%	79.7%	72.6%	34.0%	59.9%	88.0%	66.9%	10.99
W8A8	0.0%	25.6%	53.4%	50.3%	14.0%	49.5%	56.0%	35.5%	93080
ZeroQuant	0.0%*	26.0%	51.7%	49.3%	17.8%	50.9%	55.0%	35.8%	84648
LLM.int8()	74.7%	59.2%	79.7%	72.1%	34.2%	60.3%	87.0%	66.7%	11.10
Outlier Suppression	0.00%	25.8%	52.5%	48.6%	16.6%	53.4%	55.0%	36.0%	96151
SmoothQuant-O1	74.7%	59.2%	79.7%	71.2%	33.4%	58.1%	89.0%	66.5%	11.11
SmoothQuant-O2	75.0%	59.0%	79.2%	71.2%	33.0%	59.6%	88.0%	66.4%	11.14
SmoothQuant-O3	74.6%	58.9%	79.7%	71.2%	33.4%	59.9%	90.0%	66.8%	11.17

- OPT-175B : Open-Pretrained Transformer 가장 큰 모델
- Zero-shot : 학습에 사용되지 않은 정보에 대해서 정확도를 측정
- 다음 단어를 예측, 상식 추론, 이미지에 대한 정보, 주어진 정보를 활용한 대답, 텍스트간의 관계, 두 문장의 인과관계 를 평가하는 7가지 zero-shot 모델들

Experiment

1. Accuracy of zero-shot tasks with OPT models



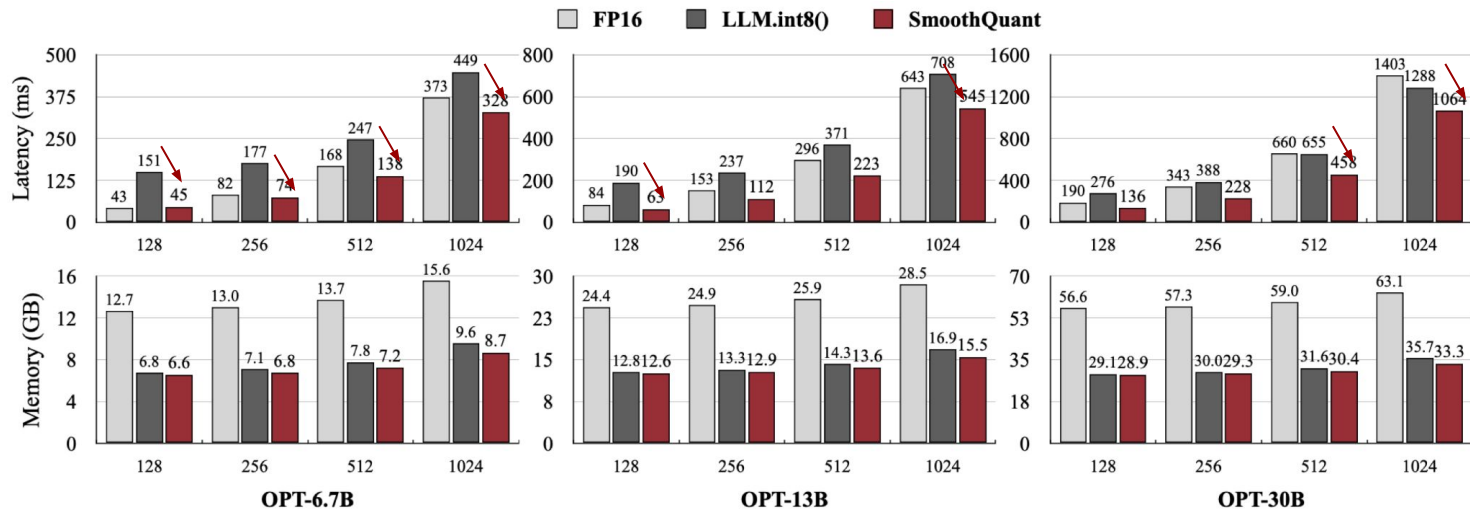
OPT-IML-30B	LAMBADA ↑	WikiText ↓
FP16	69.12%	14.26
W8A8	4.21%	576.53
ZeroQuant	5.12%	455.12
LLM.int8()	69.14%	14.27
Outlier Suppression	0.00%	9485.62
SmoothQuant-O3	69.77%	14.37

- 모델 사이즈가 작아져도 성능을 유지한다.
- FP16/LLM.int8() 은 느려진다

- Instructed-LLM에 대해서도 성능 유지
- Transformer 기반에 대해서 강함

Experiment

2. Memory Consumption in GP and latency



- X축은 시퀀스의 길이, Y축은 inference에 걸린 시간을 ms 단위로 표기
- LLM.int8() 보다 SmoothQuant가 메모리에서는 비슷하지만 속도에 있어서는 빠르다.
- 시퀀스가 두배씩 늘어남에 따라 얼마나 속도와 메모리 사용량이 변하는지에 대한 그래프
- FP16과 SmoothQuant를 비교하면 메모리 측면에서 양자화 기법이 절반을 차지한다.

Experiment

3. Perplexity scores(PPL)

- Perplexity: The number of candidates considered before generating the next token
- 다음 토큰을 예측할 때, 많은 후보군을 고려한다는 것은 성능이 좋지 않다는 것이라 PPL이 작을 수록 좋음

Wiki PPL↓	7B	13B	30B	65B
FP16	11.51	10.05	7.53	6.17
W8A8 SmoothQuant	11.56	10.08	7.56	6.20

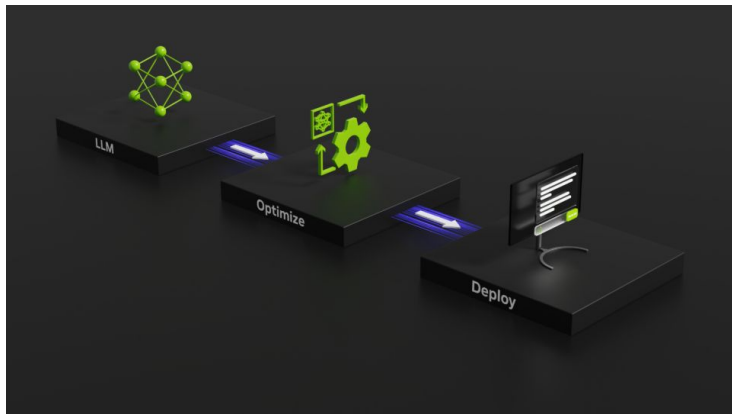
- SmoothQuant per-token quantization of activation w/ $\alpha=0.8$
- LLaMA W8A8 quantization
- per-token activation quantization and per-channel weight quantization for SmoothQuant

Model	Method	PPL	α
Llama-2-7B	FP16	5.474	0.85
	W8A8 SQ	5.515	
Llama-2-13B	FP16	4.950	0.85
	W8A8 SQ	4.929	
Llama-2-70B	FP16	3.320	0.9
	W8A8 SQ	3.359	
Falcon-7B	FP16	6.590	0.6
	W8A8 SQ	6.629	
Falcon-40B	FP16	5.228	0.7
	W8A8 SQ	5.255	
Mistral-7B	FP16	5.253	0.8
	W8A8 SQ	5.277	
Mixtral-8x7B	FP16	3.842	0.8
	W8A8 SQ	3.893	

Implementation

TensorRT-LLM

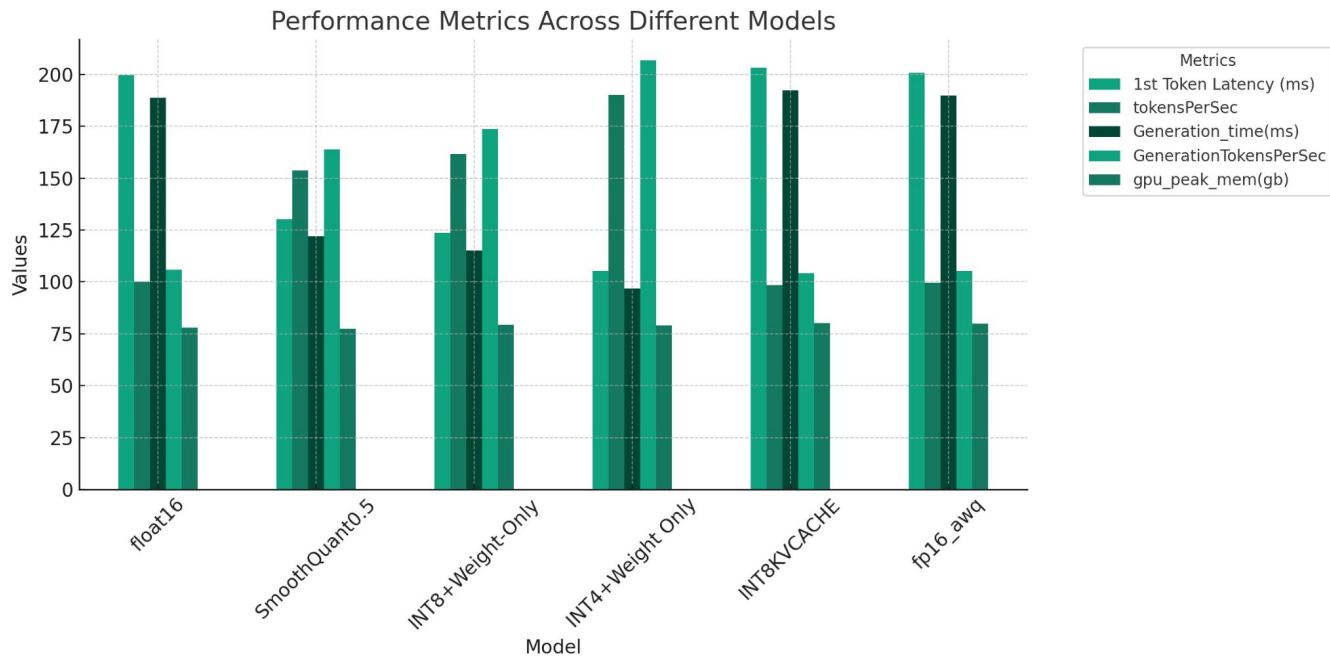
- NVIDIA의 TensorRT 엔진으로 LLM과 다른 모델을 최적화해서 NVIDIA GPU에 효과적으로 추론할 수 있게하는 Python API
- LLM 모델에 대해서 손 쉽게 여러 양자화 기법과 KV-cache 최적화, Inflight batching 등으로 LLM serving의 최적화를 쉽게 라이브러리 형식으로 적용할 수 있음
- BP16, FP8, INT8/INT4 weight only, Int8 SmoothQuant, AWQ, GPTQ, INT8+KV caching ...etc



Implementation

TensorRT-LLM

- LLaMA 2 기반의 코드 생성 오픈소스 모델 DeepSeek-coder 6.7B에 대해서 TensorRT LLM engine을 생성 후 경량화 기법에 대한 속도 성능 비교



Related Research

SmoothQuant vs QServe: W4A8KV4 Quantization(newest paper!)

Qserve

- 4-bit weights, 8-bit activations, and 4-bit KV cache
- Full support for in-flight batching & paged attention
- fused CUDA kernels for W4A8 GEMM and KV4 attention

=> SmoothQuant는 직접적으로 얼마나 activation과 weight 양자화의 밸런스를 알파를 통해 찾아야하지만, SmoothAttention에서 Key만 스케일하여 블록 중간의 활성화를 달성할 수 있다.

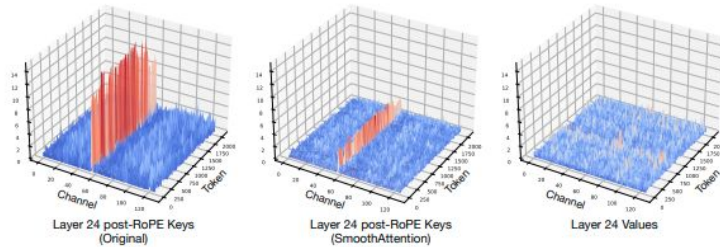


Fig. 7: SmoothAttention effectively smooths the outliers in Keys. Values doesn't suffer from outliers.

Related Research

SmoothQuant vs QServe: W4A8KV4 Quantization(newest paper!)

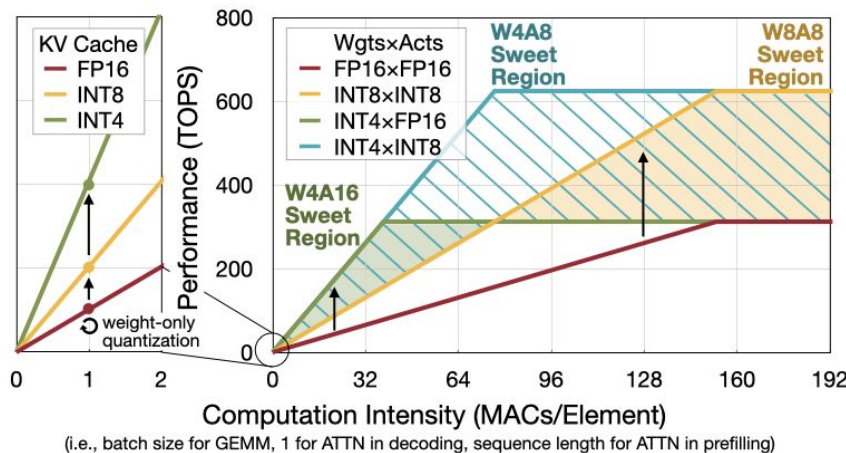
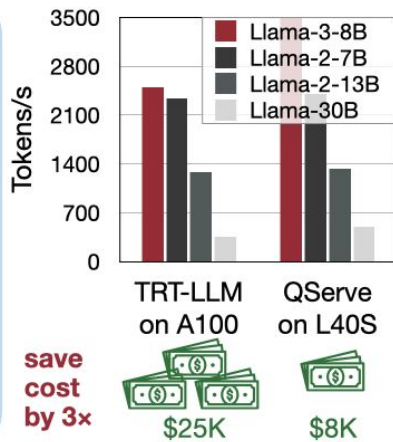
- Fully Pytorch based user interface w/ TRT-LLM level efficiency and Pytorch level flexibility
- maximum achievable **serving throughput** of Llama-3-8B by 1.2× on A100, 1.4× on L40S compared to TensorRT-LLM
- efficiency of LLM serving on GPUs is critically influenced by operations on **low-throughput CUDA cores**.

QoQ Algorithm

- Progressive Group Quantization
- SmoothAttention
- Activation-aware Channel Reordering

QServe System

- Compute-aware Weight Reordering
- Efficient Dequantization (Mul → Sub)
- Make Fused Attention Memory-bound



Conclusion

- LLM에 8비트 가중치, 8비트 활성화 (W8A8) 양자화를 가능하게 하는 PTQ 방법
- 수학적으로 동등한 변환을 통해 양자화과제를 활성화에서 가중치를 옮기면서 활성화의 이상값 (outlier)을 해결하는 접근 방식
- SmoothQuant 내의 매개변수를 수동으로 설정하는 것이 어려워, 이미지 특성에 적응하는 것에는 성능이 제한되어 튜닝 파이프라인으로 모델 레이어에 대한 파라미터 설정이 필요

What's coming Next

- 1-bit LLMs : $(-1, 0, 1)$ 세개로 모델의 파라미터를 표현하여 효율적인 결과를 보여줌
- Multi-modal LLM Quantization
- GPU process based LLM Quantization and serving pipeline